Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются. За частично верное решение начисляются баллы.

Задача 1. Удалить заданную цифру

2 балла

Напишите функцию `remove_digit(number, digit)`.

`number` - целое неотрицательное число. `digit` - целое число от 0 до 9. Удалите из записи `number` все вхождения цифры `digit`.

Порядок оставшихся цифр должен совпадать с их порядком в исходном числе. Верните результат как целое число: начальные нули в нём не сохраняются. Если ни одной цифры не осталось, верните `0`. Число `0` считайте записью из одной цифры `0`.

Преобразовывать число или отдельные цифры в строки нельзя.

```
def remove_digit(number: int, digit: int) -> int:
    ...
```

Примеры:

```
assert remove_digit(120203, 0) == 1223
assert remove_digit(120203, 2) == 1003
assert remove_digit(555, 5) == 0
assert remove_digit(1002, 1) == 2
assert remove_digit(0, 0) == 0
```

Задача 2. Соседние слова одинаковой длины

2 балла

Напишите функцию `equal_length_neighbors(text)`.

Верните список пар соседних слов, для которых длины слов равны. Каждая пара должна быть кортежем `(первое слово, второе слово)` в порядке их появления в строке.

Проверяются все соседние пары исходной строки. Одно слово может входить в две пары. Сохраните повторы и исходный регистр букв.

Строка содержит только буквы и пробелы. Пробелов между словами, в начале и конце строки может быть несколько. Если подходящих пар нет или слов меньше двух, верните пустой список.

```
def equal_length_neighbors(text: str) -> list[tuple[str, str]]:
    ...
```

Примеры:

```
assert equal_length_neighbors('кот кит и мы ты') == [('кот', 'кит'), ('мы', 'ты')]
assert equal_length_neighbors('дом кот кит') == [('дом', 'кот'), ('кот', 'кит')]
assert equal_length_neighbors('я читаю') == []
assert equal_length_neighbors('  Кот   Кит  ') == [('Кот', 'Кит')]
assert equal_length_neighbors('Привет') == []
assert equal_length_neighbors('   ') == []
assert equal_length_neighbors('') == []
```

Задача 3. Первый блок чётных чисел

2,5 балла

Напишите функцию `first_even_run(values, k)`.

`values` - список целых чисел, `k` - целое неотрицательное число.

Верните новый список из первого непрерывного блока длины `k`, в котором все числа чётные. Если таких блоков несколько, выберите тот, который начинается раньше. Элементы блока должны идти подряд в исходном списке: пропускать неподходящие элементы внутри блока нельзя.

Если такого блока нет, верните пустой список. При `k == 0` также верните пустой список и не просматривайте элементы. Как только первый подходящий блок найден, просмотр исходного списка должен прекратиться.

```
def first_even_run(values: list[int], k: int) -> list[int]:
    ...
```

Примеры:

```
assert first_even_run([2, 7, 4, 6, 8], 2) == [4, 6]
assert first_even_run([2, 4, 6], 2) == [2, 4]
assert first_even_run([2, 1, 4], 2) == []
assert first_even_run([-2, 0, 3], 2) == [-2, 0]
assert first_even_run([], 2) == []
assert first_even_run([1, -1, 0], 0) == []
```

Задача 4. Продажи разрешённых товаров

3,5 балла

Напишите функцию `count_allowed_sales(sales, banned)`.

`sales` - список названий проданных товаров, `banned` - список названий товаров, которые нужно исключить из отчёта.

Верните словарь: ключ - товар, значение - число его вхождений в `sales`, если его нет в `banned`. Здесь нужно учитывать все повторения в первом списке. Повторы во втором списке ничего не меняют. Значения, встречающиеся только во втором списке, в результат не включайте.

Оба списка содержат строки. Строки сравниваются точно, с учётом регистра; очищать или преобразовывать их не нужно. Порядок ключей не важен. Если после исключения ничего не осталось, верните пустой словарь.

Требование к эффективности: среднее время работы `O(n + m)`, включая подготовку данных, где `n` и `m` - длины двух входных списков. Считайте, что проверка наличия строки в словаре или множестве, добавление записи в словарь и добавление строки в множество в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def count_allowed_sales(sales: list[str], banned: list[str]) -> dict[str, int]:
    ...
```

Примеры:

```
assert count_allowed_sales(
    ['чай', 'хлеб', 'чай', 'сок', 'хлеб'],
    ['хлеб'],
) == {'чай': 2, 'сок': 1}
assert count_allowed_sales(['чай', 'чай'], ['чай', 'чай']) == {}
assert count_allowed_sales(['чай', 'чай'], []) == {'чай': 2}
assert count_allowed_sales([], ['нет записей']) == {}
```

Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются, но если нужны для иллюстрации идеи - пишите. За частично верное решение начисляются баллы.

Задача 1. Оставить цифры не меньше порога

2 балла

Напишите функцию `keep_digits_at_least(number, threshold)`.

`number` - целое неотрицательное число. `threshold` - целое число от 0 до 9. Оставьте в записи `number` только цифры, которые не меньше `threshold`.

Порядок оставшихся цифр должен совпадать с их порядком в исходном числе. Верните результат как целое число: начальные нули в нём не сохраняются. Если ни одной цифры не осталось, верните `0`. Число `0` считайте записью из одной цифры `0`.

Преобразовывать число или отдельные цифры в строки нельзя.

```
def keep_digits_at_least(number: int, threshold: int) -> int:
    ...
```

Примеры:

```
assert keep_digits_at_least(58216, 5) == 586
assert keep_digits_at_least(10203, 2) == 23
assert keep_digits_at_least(123, 8) == 0
assert keep_digits_at_least(1002, 0) == 1002
assert keep_digits_at_least(0, 0) == 0
```

Задача 2. Следующее слово длиннее

2 балла

Напишите функцию `growing_neighbors(text)`.

Верните список пар соседних слов, для которых длина второго слова строго больше длины первого. Каждая пара должна быть кортежем `(первое слово, второе слово)` в порядке их появления в строке.

Проверяются все соседние пары исходной строки. Одно слово может входить в две пары. Сохраните повторы и исходный регистр букв.

Строка содержит только буквы и пробелы. Пробелов между словами, в начале и конце строки может быть несколько. Если подходящих пар нет или слов меньше двух, верните пустой список.

```
def growing_neighbors(text: str) -> list[tuple[str, str]]:
    ...
```

Примеры:

```
assert growing_neighbors('я вижу дом далеко') == [('я', 'вижу'), ('дом', 'далеко')]
assert growing_neighbors('я мы кот') == [('я', 'мы'), ('мы', 'кот')]
assert growing_neighbors('кот кит') == []
assert growing_neighbors('  Кот   Кит  ') == []
assert growing_neighbors('Привет') == []
assert growing_neighbors('   ') == []
assert growing_neighbors('') == []
```

Задача 3. Первый блок нечётных чисел

2,5 балла

Напишите функцию `first_odd_run(values, k)`.

`values` - список целых чисел, `k` - целое неотрицательное число.

Верните новый список из первого непрерывного блока длины `k`, в котором все числа нечётные. Если таких блоков несколько, выберите тот, который начинается раньше. Элементы блока должны идти подряд в исходном списке: пропускать неподходящие элементы внутри блока нельзя.

Если такого блока нет, верните пустой список. При `k == 0` также верните пустой список и не просматривайте элементы. Как только первый подходящий блок найден, просмотр исходного списка должен прекратиться.

```
def first_odd_run(values: list[int], k: int) -> list[int]:
    ...
```

Примеры:

```
assert first_odd_run([1, 4, 3, 5, 7], 2) == [3, 5]
assert first_odd_run([1, 3, 5], 2) == [1, 3]
assert first_odd_run([1, 2, 3], 2) == []
assert first_odd_run([-3, -1, 0], 2) == [-3, -1]
assert first_odd_run([], 2) == []
assert first_odd_run([1, -1, 0], 0) == []
```

Задача 4. Посещения без сотрудников

3,5 балла

Напишите функцию `count_guest_visits(visits, staff)`.

`visits` - список имён посетителей в журнале входа, `staff` - список имён сотрудников. В отчёте нужно учитывать только посетителей, которые не являются сотрудниками.

Верните словарь: ключ - посетитель, значение - число его вхождений в `visits`, если его нет в `staff`. Здесь нужно учитывать все повторения в первом списке. Повторы во втором списке ничего не меняют. Значения, встречающиеся только во втором списке, в результат не включайте.

Оба списка содержат строки. Строки сравниваются точно, с учётом регистра; очищать или преобразовывать их не нужно. Порядок ключей не важен. Если после исключения ничего не осталось, верните пустой словарь.

Требование к эффективности: среднее время работы `O(n + m)`, включая подготовку данных, где `n` и `m` - длины двух входных списков. Считайте, что проверка наличия строки в словаре или множестве, добавление записи в словарь и добавление строки в множество в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def count_guest_visits(visits: list[str], staff: list[str]) -> dict[str, int]:
    ...
```

Примеры:

```
assert count_guest_visits(
    ['Аня', 'Борис', 'Аня', 'Вера', 'Борис'],
    ['Борис'],
) == {'Аня': 2, 'Вера': 1}
assert count_guest_visits(['Аня', 'Аня'], ['Аня', 'Аня']) == {}
assert count_guest_visits(['Аня', 'Аня'], []) == {'Аня': 2}
assert count_guest_visits([], ['нет записей']) == {}
```

Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются. За частично верное решение начисляются баллы.

Задача 1. Оставить цифры не больше порога

2 балла

Напишите функцию `keep_digits_at_most(number, threshold)`.

`number` - целое неотрицательное число. `threshold` - целое число от 0 до 9. Оставьте в записи `number` только цифры, которые не больше `threshold`.

Порядок оставшихся цифр должен совпадать с их порядком в исходном числе. Верните результат как целое число: начальные нули в нём не сохраняются. Если ни одной цифры не осталось, верните `0`. Число `0` считайте записью из одной цифры `0`.

Преобразовывать число или отдельные цифры в строки нельзя.

```
def keep_digits_at_most(number: int, threshold: int) -> int:
    ...
```

Примеры:

```
assert keep_digits_at_most(58216, 5) == 521
assert keep_digits_at_most(90802, 2) == 2
assert keep_digits_at_most(987, 3) == 0
assert keep_digits_at_most(1002, 9) == 1002
assert keep_digits_at_most(0, 0) == 0
```

Задача 2. Следующее слово короче

2 балла

Напишите функцию `shrinking_neighbors(text)`.

Верните список пар соседних слов, для которых длина второго слова строго меньше длины первого. Каждая пара должна быть кортежем `(первое слово, второе слово)` в порядке их появления в строке.

Проверяются все соседние пары исходной строки. Одно слово может входить в две пары. Сохраните повторы и исходный регистр букв.

Строка содержит только буквы и пробелы. Пробелов между словами, в начале и конце строки может быть несколько. Если подходящих пар нет или слов меньше двух, верните пустой список.

```
def shrinking_neighbors(text: str) -> list[tuple[str, str]]:
    ...
```

Примеры:

```
assert shrinking_neighbors('далеко дом вижу я') == [('далеко', 'дом'), ('вижу', 'я')]
assert shrinking_neighbors('кот мы я') == [('кот', 'мы'), ('мы', 'я')]
assert shrinking_neighbors('кот кит') == []
assert shrinking_neighbors('  Кот   Кит  ') == []
assert shrinking_neighbors('Привет') == []
assert shrinking_neighbors('   ') == []
assert shrinking_neighbors('') == []
```

Задача 3. Первый блок положительных чисел

2,5 балла

Напишите функцию `first_positive_run(values, k)`.

`values` - список целых чисел, `k` - целое неотрицательное число.

Верните новый список из первого непрерывного блока длины `k`, в котором все числа строго положительные. Если таких блоков несколько, выберите тот, который начинается раньше. Элементы блока должны идти подряд в исходном списке: пропускать неподходящие элементы внутри блока нельзя.

Если такого блока нет, верните пустой список. При `k == 0` также верните пустой список и не просматривайте элементы. Как только первый подходящий блок найден, просмотр исходного списка должен прекратиться.

```
def first_positive_run(values: list[int], k: int) -> list[int]:
    ...
```

Примеры:

```
assert first_positive_run([1, 0, 3, 5, 7], 2) == [3, 5]
assert first_positive_run([1, 3, 5], 2) == [1, 3]
assert first_positive_run([1, -2, 3], 2) == []
assert first_positive_run([-3, 0, 2], 1) == [2]
assert first_positive_run([], 2) == []
assert first_positive_run([1, -1, 0], 0) == []
```

Задача 4. Выдачи книг без учебников

3,5 балла

Напишите функцию `count_book_loans(loans, textbooks)`.

`loans` - список названий книг в журнале выдач, `textbooks` - список названий учебников. Учебники нужно исключить из отчёта.

Верните словарь: ключ - книга, значение - число его вхождений в `loans`, если его нет в `textbooks`. Здесь нужно учитывать все повторения в первом списке. Повторы во втором списке ничего не меняют. Значения, встречающиеся только во втором списке, в результат не включайте.

Оба списка содержат строки. Строки сравниваются точно, с учётом регистра; очищать или преобразовывать их не нужно. Порядок ключей не важен. Если после исключения ничего не осталось, верните пустой словарь.

Требование к эффективности: среднее время работы `O(n + m)`, включая подготовку данных, где `n` и `m` - длины двух входных списков. Считайте, что проверка наличия строки в словаре или множестве, добавление записи в словарь и добавление строки в множество в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def count_book_loans(loans: list[str], textbooks: list[str]) -> dict[str, int]:
    ...
```

Примеры:

```
assert count_book_loans(
    ['Дюна', 'Алгебра', 'Дюна', 'Солярис', 'Алгебра'],
    ['Алгебра'],
) == {'Дюна': 2, 'Солярис': 1}
assert count_book_loans(['Дюна', 'Дюна'], ['Дюна', 'Дюна']) == {}
assert count_book_loans(['Дюна', 'Дюна'], []) == {'Дюна': 2}
assert count_book_loans([], ['нет записей']) == {}
```

Фамилия, имя: _____ Группа: _____

Материал: лекции 1, 2, 3, 5 и 6. Импорты и внешние библиотеки не нужны.

Работа выполняется на бумаге, без запуска кода. В каждой задаче напишите функцию на Python. Читать данные через `input` и печатать результат не нужно: функция принимает аргументы и возвращает результат. Аннотации типов необязательны. Можно использовать встроенные функции и методы Python. Исходные списки менять нельзя.

Задачи независимы, их можно решать в любом порядке. Письменные объяснения и ручная трассировка не требуются. За частично верное решение начисляются баллы.

Задача 1. Оставить цифры заданной чётности

2 балла

Напишите функцию `keep_digit_parity(number, parity)`.

`number` - целое неотрицательное число. `parity` равен `0` или `1`. При `parity == 0` оставьте только чётные цифры, при `parity == 1` - только нечётные.

Порядок оставшихся цифр должен совпадать с их порядком в исходном числе. Верните результат как целое число: начальные нули в нём не сохраняются. Если ни одной цифры не осталось, верните `0`. Число `0` считайте записью из одной цифры `0`.

Преобразовывать число или отдельные цифры в строки нельзя.

```
def keep_digit_parity(number: int, parity: int) -> int:
    ...
```

Примеры:

```
assert keep_digit_parity(58216, 0) == 826
assert keep_digit_parity(58216, 1) == 51
assert keep_digit_parity(135, 0) == 0
assert keep_digit_parity(1002, 0) == 2
assert keep_digit_parity(0, 0) == 0
```

Задача 2. Соседние слова разной длины

2 балла

Напишите функцию `different_length_neighbors(text)`.

Верните список пар соседних слов, для которых длины слов различаются. Каждая пара должна быть кортежем `(первое слово, второе слово)` в порядке их появления в строке.

Проверяются все соседние пары исходной строки. Одно слово может входить в две пары. Сохраните повторы и исходный регистр букв.

Строка содержит только буквы и пробелы. Пробелов между словами, в начале и конце строки может быть несколько. Если подходящих пар нет или слов меньше двух, верните пустой список.

```
def different_length_neighbors(text: str) -> list[tuple[str, str]]:
    ...
```

Примеры:

```
assert different_length_neighbors('кот кит и мы ты') == [('кит', 'и'), ('и', 'мы')]
assert different_length_neighbors('я мы кот') == [('я', 'мы'), ('мы', 'кот')]
assert different_length_neighbors('кот кит') == []
assert different_length_neighbors('  Кот   Кит  ') == []
assert different_length_neighbors('Привет') == []
assert different_length_neighbors('   ') == []
assert different_length_neighbors('') == []
```

Задача 3. Первый блок отрицательных чисел

2,5 балла

Напишите функцию `first_negative_run(values, k)`.

`values` - список целых чисел, `k` - целое неотрицательное число.

Верните новый список из первого непрерывного блока длины `k`, в котором все числа строго отрицательные. Если таких блоков несколько, выберите тот, который начинается раньше. Элементы блока должны идти подряд в исходном списке: пропускать неподходящие элементы внутри блока нельзя.

Если такого блока нет, верните пустой список. При `k == 0` также верните пустой список и не просматривайте элементы. Как только первый подходящий блок найден, просмотр исходного списка должен прекратиться.

```
def first_negative_run(values: list[int], k: int) -> list[int]:
    ...
```

Примеры:

```
assert first_negative_run([-1, 0, -3, -5, -7], 2) == [-3, -5]
assert first_negative_run([-1, -3, -5], 2) == [-1, -3]
assert first_negative_run([-1, 2, -3], 2) == []
assert first_negative_run([3, 0, -2], 1) == [-2]
assert first_negative_run([], 2) == []
assert first_negative_run([1, -1, 0], 0) == []
```

Задача 4. Ошибки без игнорируемых кодов

3,5 балла

Напишите функцию `count_errors(errors, ignored)`.

`errors` - список кодов ошибок в журнале, `ignored` - список кодов, которые нужно исключить из отчёта. Коды ошибок - строки.

Верните словарь: ключ - код ошибки, значение - число его вхождений в `errors`, если его нет в `ignored`. Здесь нужно учитывать все повторения в первом списке. Повторы во втором списке ничего не меняют. Значения, встречающиеся только во втором списке, в результат не включайте.

Оба списка содержат строки. Строки сравниваются точно, с учётом регистра; очищать или преобразовывать их не нужно. Порядок ключей не важен. Если после исключения ничего не осталось, верните пустой словарь.

Требование к эффективности: среднее время работы `O(n + m)`, включая подготовку данных, где `n` и `m` - длины двух входных списков. Считайте, что проверка наличия строки в словаре или множестве, добавление записи в словарь и добавление строки в множество в среднем выполняются за `O(1)`. Объяснять сложность письменно не нужно.

```
def count_errors(errors: list[str], ignored: list[str]) -> dict[str, int]:
    ...
```

Примеры:

```
assert count_errors(['E1', 'E2', 'E1', 'E3', 'E2'], ['E2']) == {'E1': 2, 'E3': 1}
assert count_errors(['E1', 'E1'], ['E1', 'E1']) == {}
assert count_errors(['E1', 'E1'], []) == {'E1': 2}
assert count_errors([], ['нет записей']) == {}
```